

NexiLock

Plan de tests d'intégration et d'acceptation

BTS SIO option SLAM, Emma CLUZET, CFA Doranco, Session 2026

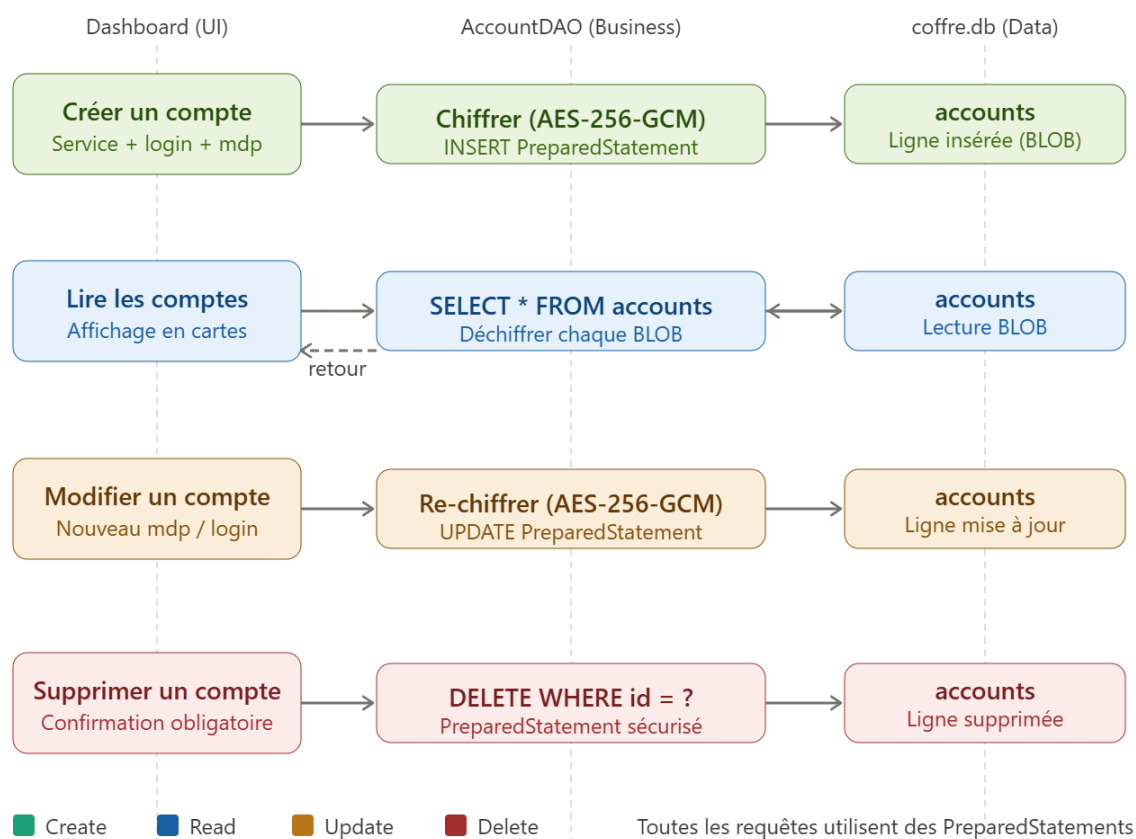
À quoi sert ce document ?	1
Tests unitaires : DatabaseManager	1
Tests unitaires : CryptoService	2
Tests d'intégration : AccountDAO	4
Tests d'acceptation : Vérification manuelle.....	4

À quoi sert ce document ?

Ce document décrit les tests que j'ai rédigés et exécutés pour vérifier que NexiLock fonctionne correctement. Il y a trois niveaux de tests :

- Tests unitaires : je teste chaque classe séparément, sans lancer toute l'application.
- Tests d'intégration : je teste plusieurs classes qui travaillent ensemble (par exemple AccountDAO + CryptoService + la base de données).
- Tests d'acceptation : je vérifie manuellement que l'application répond aux besoins décrits dans le cahier des charges.

Élément	Valeur
Outil utilisé	JUnit 5.10 avec Maven Surefire 3.2.5
Commande d'exécution	mvn clean test (dans le dossier du projet)
Base de test	Base SQLite temporaire créée et supprimée automatiquement et ne touche pas à coffre.db
Rédactrice	Emma CLUZET
Date d'exécution	Semaine du 24 au 28 février 2026 (Sprint 4)



Tests unitaires : DatabaseManager

```

@BeforeAll
static void setup() {
    // Préparation de l'environnement de test
    DatabaseManager.initDatabase();
}

@Test
void testFileExists() {
    // Vérifie que le fichier physique 'coffre.db' a bien été créé
    File dbFile = new File("coffre.db");
    assertTrue(dbFile.exists(), "Le fichier coffre.db devrait être créé à la racine.");
}

```

```

@Test
void testTablesExist() {
    // Vérifie que les tables 'master' et 'accounts' sont présentes
    try (Connection conn = DriverManager.getConnection(URL)) {
        DatabaseMetaData meta = conn.getMetaData();

        // Vérification de la table 'accounts'
        try (ResultSet rsAccounts = meta.getTables(null, null, "accounts", null)) {
            assertTrue(rsAccounts.next(), "La table 'accounts' n'existe pas en base.");
        }

        // Vérification de la table 'master'
        try (ResultSet rsMaster = meta.getTables(null, null, "master", null)) {
            assertTrue(rsMaster.next(), "La table 'master' n'existe pas en base.");
        }
    } catch (Exception e) {
        fail("Erreur lors de l'accès aux métadonnées de la base : " + e.getMessage());
    }
}

```

Ces tests vérifient que la classe DatabaseManager crée bien la base de données et les tables au démarrage. J'utilise un fichier temporaire pour éviter de toucher au vrai coffre.db pendant les tests.

ID	Ce que je teste	Ce que je donne	Ce que j'attends	Résultat	OK ?
UT-DB-01	Le fichier de base est créé	Appel initDatabase()	Fichier temporaire présent sur le disque	Fichier créé	PASS
UT-DB-02	La table accounts existe	Lecture des métadonnées JDBC	Table accounts trouvée	Table présente	PASS
UT-DB-03	La table master existe	Lecture des métadonnées JDBC	Table master trouvée	Table présente	PASS
UT-DB-04	Les colonnes de accounts sont correctes	getColumns() sur accounts	id, service, username, password_enc	Colonnes conformes	PASS

UT-DB-05	Les colonnes de master sont correctes	getColumn() sur master	id, salt, hash	Colonnes conformes	PASS
UT-DB-06	Appeler initDatabase() deux fois ne plante pas	initDatabase() × 2	Pas d'exception (IF NOT EXISTS)	Pas d'erreur	PASS

Tests unitaires : CryptoService

```
@Test
@Order(3)
void testEncryptDecryptRoundTrip() throws Exception {
    String original = "monSuperMotDePasse@2026!";
    byte[] blob = crypto.encrypt(original);
    String decrypted = crypto.decrypt(blob);
    assertEquals(original, decrypted, "Le texte déchiffré doit être identique à l'original.");
}
```

Ces tests vérifient le moteur de chiffrement. C'est la partie la plus critique de NexiLock : si CryptoService ne fonctionne pas correctement, les mots de passe ne sont pas bien protégés.

ID	Ce que je teste	Ce que je donne	Ce que j'attends	OK ?
UT-CR-01	Déverrouillage avec le bon mot de passe	Mot de passe correct	CryptoService créé (non null)	PASS
UT-CR-02	Mauvais mot de passe → refus	Mauvais mot de passe	Retourne null — pas d'exception	PASS
UT-CR-03	Chiffrer puis déchiffrer donne le même texte	Texte clair quelconque	Texte déchiffré = texte original	PASS
UT-CR-04	Deux chiffrements du même texte donnent deux BLOBs différents	Même texte × 2	Les deux BLOBs sont différents (IV aléatoire)	PASS
UT-CR-05	Le BLOB contient bien le vecteur d'initialisation (12 octets)	Chiffrement d'un texte	Taille du BLOB > 12 octets	PASS
UT-CR-06	Déchiffrer avec la mauvaise clé lève une exception	Clé d'un autre coffre	Exception levée	PASS
UT-CR-07	Vérification du mot de passe correct	Bon MDP + hash stocké	Retourne true	PASS
UT-CR-08	Vérification d'un mauvais mot de passe	Mauvais MDP + hash stocké	Retourne false	PASS
UT-CR-09	Le générateur produit la bonne longueur	length = 8, 16 ou 32	Mot de passe de exactement length caractères	PASS
UT-CR-10	Le mot de passe généré contient 4 types de caractères	length = 16	Au moins 1 maj, 1 min, 1 chiffre, 1 spécial	PASS

UT-CR-11	Longueur < 8 est refusée	length = 7	IllegalArgumentException	PASS
UT-CR-12	Longueur > 32 est refusée	length = 33	IllegalArgumentException	PASS
UT-CR-13	Deux mots de passe générés sont différents	Deux appels	Valeurs différentes	PASS
UT-CR-14	La Recovery Key a le bon format	generateRecoveryKey()	Format XXXXXX-XXXXXX-XXXXXX-XXXXXX	PASS
UT-CR-15	Deux Recovery Keys générées sont différentes	Deux appels	Valeurs différentes	PASS

Tests d'intégration : AccountDAO

```

@Test
@Order(3)
void testPasswordIsStoredEncrypted() throws Exception {
    // Vérification directe en base : le BLOB ne doit pas contenir le mot de passe en clair
    try (var conn = DatabaseManager.getConnection();
         var ps = conn.prepareStatement("SELECT password_enc FROM accounts WHERE id=?")) {
        ps.setInt(1, insertedId);
        try (var rs = ps.executeQuery()) {
            assertTrue(rs.next());
            byte[] blob = rs.getBytes("password_enc");
            String blobStr = new String(blob);
            assertFalse(blobStr.contains("GhPass@123"),
                "Le mot de passe ne doit jamais apparaître en clair dans la base.");
        }
    }
}

```

Ces tests vérifient que la couche d'accès aux données fonctionne bien avec le moteur de chiffrement et la base SQLite ensemble. J'utilise une base temporaire pour ne pas polluer les données réelles.

ID	Ce que je teste	Ce que je donne	Ce que j'attends	OK ?
IT-DAO-01	Insérer un compte → id auto-généré	Nouveau compte (service, login, MDP)	account.getId() > 0	PASS
IT-DAO-02	Récupérer un compte → mot de passe déchiffré correct	Base avec 1 compte	MDP déchiffré = MDP saisi à l'insertion	PASS
IT-DAO-03	Le mot de passe n'est pas en clair dans la base	SELECT direct sur le BLOB	Le BLOB ne contient pas le MDP en clair	PASS
IT-DAO-04	Modifier un compte → nouvelles valeurs récupérées	Compte existant modifié	findAll() retourne les nouvelles valeurs	PASS

IT-DAO-05	Recherche filtre correctement (insensible à la casse)	Recherche 'git'	GitHub et GitLab trouvés, pas Gmail	PASS
IT-DAO-06	Supprimer un compte → il n'est plus dans la liste	id existant	findAll().size() diminue de 1	PASS

Tests d'acceptation : Vérification manuelle

Ces tests sont exécutés à la main, en lançant l'application et en vérifiant que tout fonctionne du point de vue de l'utilisateur. Ils confirment que les besoins du cahier des charges sont bien satisfaits.

ID	Scénario testé	Ce que je fais	Ce que j'attends	OK ?
AT-01	Lancement de l'application	mvn clean javafx:run	La fenêtre de connexion s'ouvre en moins de 3 secondes	PASS
AT-02	Connexion avec le bon mot de passe Windows	Je saisis mon vrai mot de passe Windows	Le tableau de bord s'ouvre	PASS
AT-03	Connexion avec un mauvais mot de passe	Je saisis un mot de passe incorrect	Un message d'erreur s'affiche, l'accès est refusé	PASS
AT-04	Bascule automatique vers Recovery Key	Je simule un échec JNA (débranchement réseau domaine)	Le champ Recovery Key apparaît à la place du mot de passe	PASS
AT-05	Déverrouillage avec la Recovery Key	Je saisis la bonne Recovery Key	Le tableau de bord s'ouvre normalement	PASS
AT-06	Ajouter un nouveau compte	Je remplis le formulaire et clique Enregistrer	Le compte apparaît immédiatement dans la liste	PASS
AT-07	Modifier un compte existant	Je sélectionne un compte, je change le mot de passe, je clique Modifier	Les nouvelles valeurs s'affichent dans la carte	PASS
AT-08	Supprimer un compte avec confirmation	Je clique Supprimer et je confirme	Le compte disparaît de la liste	PASS
AT-09	Générer un mot de passe de 16 caractères	Je règle le slider sur 16 et clique Générer	Le champ mot de passe se remplit avec 16 caractères	PASS
AT-10	Rechercher un compte	Je tape 'git' dans la barre de recherche	Seuls les comptes GitHub/GitLab s'affichent	PASS
AT-11	Vérifier que le MDP n'est pas en clair dans coffre.db	J'ouvre coffre.db avec DB Browser for SQLite	La colonne password_enc contient du texte chiffré illisible	PASS
AT-12	Premier lancement — affichage de la Recovery Key	Je supprime coffre.db et relance l'application	Une boîte de dialogue affiche la Recovery Key générée	PASS